Chapter 6: Data Models

Welcome back! In our previous chapter, we successfully set up our backend application. We got our server running, connected it to a database (MongoDB using Mongoose), and prepared it to receive requests. That's like building the frame of a house and connecting the plumbing!

But now, we have an empty house (database). How do we tell it what kind of furniture goes where, or how many rooms there should be? How do we define the *structure* of the information we want to store, like user accounts or detailed interview reports?

This is where **Data Models** come in.

What Problem Are We Solving?

Imagine our database as a vast, empty warehouse. If we just start throwing items (data) into it without any organization, it would be a chaotic mess! We'd never find anything, and we wouldn't know what each item is supposed to look like.

Data Models are like blueprints or a catalog for our database. They define the exact structure, rules, and relationships for different types of information we want to store.

For our interview-ai-yt application, we need to store two main types of important data:

1. **Users:** From our User Authentication System, we need to store details like a user's username, email, and password.
2. **Interview Reports:** From our Interview AI Service, we need to store complex information like the jobDescription, matchScore, technicalQuestions, behavioralQuestions, and skillGaps for each report generated.

Without Data Models, our database wouldn't know that a "User" should have an email or that an "Interview Report" needs a matchScore that's a number between 0 and 100. Data Models ensure our data is organized, consistent, and easy to find, save, or update.

Key Concepts: Your Data Blueprint Tools

To create these blueprints for our database using Mongoose (the tool we use to talk to MongoDB), we use a few key ideas:

1. **Schema (The Blueprint Itself):** This is the core definition. A "Schema" is like a detailed drawing that specifies the names of all the fields (like username), what type of data each field holds (like a String or Number), and any special rules (like if a field is required or must be unique).
2. **Model (The Factory):** Once we have a Schema (blueprint), we use it to create a "Model." A Model is like a factory that produces and manages all the "documents" (individual records, like one specific user or one specific interview report) that follow that blueprint. It gives us functions to create new records, find existing ones, update them, or delete them.
3. **Fields/Properties:** These are the individual pieces of information within a record. For a "User," username and email are fields.
4. **Data Types:** Each field needs a specific type, like String (for text), Number (for scores or counts), Boolean (for true/false), Date (for timestamps), or even Array (for lists of things, like multiple questions).
5. **Validation:** These are the rules we put on our fields to ensure data quality. Examples include required: true (a field *must* have a value), unique: true (no two records can have the same value for this field), min, max, or enum (a field must be one of a predefined list of values).
6. **References (Relationships):** Often, different types of data are connected. For example, an Interview Report *belongs to* a User. Data Models allow us to create these "references," linking one piece of data to another, much like a library catalog might link a book to its author.

How to Use Data Models

As developers, we define our Data Models once at the beginning. These definitions then become the foundation for how our backend application interacts with the database.

For example, when our User Authentication System creates a new user, it will use the userModel we define. Similarly, when our Interview AI Service generates a report, it will use the interviewReportModel to save that report.

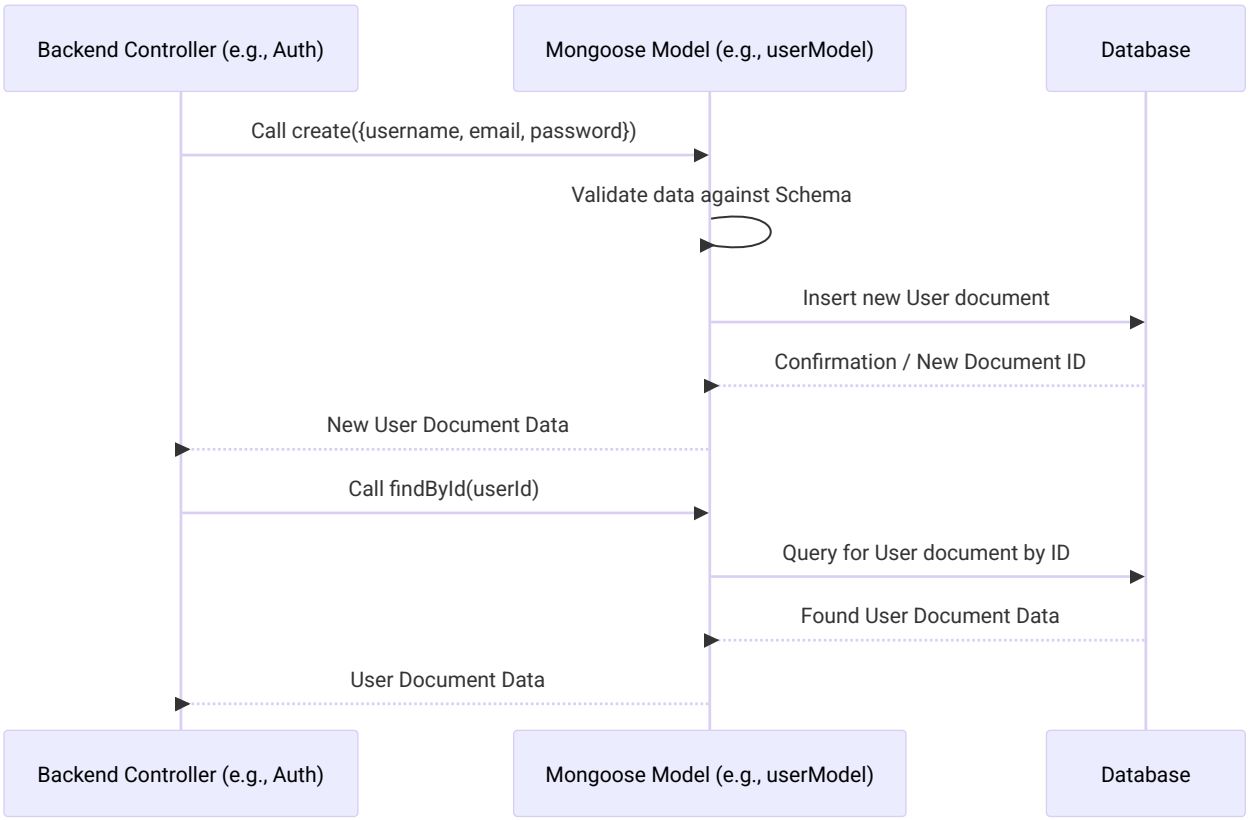
You don't "use" the model in the sense of clicking a button; you *define* it in code, and then your other backend code uses the defined model to perform database operations.

Under the Hood: How Your Blueprints Guide the Database

Let's see the sequence of how a Data Model helps manage your information.

1. **You define a Schema:** You write code to tell Mongoose what a "User" or an "Interview Report" should look like.
2. **Mongoose creates a Model:** Mongoose takes your Schema and creates a "Model" object. This Model is now your primary tool for interacting with the database for that specific type of data.
3. **Backend Code Uses the Model:** When your backend needs to save a new user, it calls userModel.create(...). When it needs to fetch an interview report, it calls interviewReportModel.findById(...).
4. **Model Talks to MongoDB:** The Mongoose Model takes your request, applies the rules from the Schema (e.g., checks if required fields are present, validates data types), and then sends a command to the actual MongoDB database to save or retrieve the data.
5. **MongoDB Stores/Returns Data:** MongoDB then saves the data in the defined structure or returns the requested data back through the Mongoose Model.

Here's a simple diagram to visualize this:



Diving Deeper into the Code

Let's look at the actual code in our Backend/src/models/ folder to see how these blueprints are defined.

1. **The User Model (Backend/src/models/user.model.js)**

This file defines the blueprint for our User data. It specifies that each user will have a username, email, and password.

```
// Backend/src/models/user.model.js
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
    username: {
        type: String,
        unique: [true, "username already taken"], // Rule: must be unique
        required: true, // Rule: must have a value
    },
    email: {
        type: String,
        unique: [true, "Account already exists with this email address"],
        required: true,
    },
    password: {
        type: String,
        required: true
    }
});

// Create the 'User' model from the schema
const userModel = mongoose.model("users", userSchema);

module.exports = userModel; // Make it available to other parts of our app
```

Explanation:

- new mongoose.Schema({...}) creates our blueprint for a User.
- Each field like username, email, password has a type (e.g., String).
- unique: true means no two users can have the same username or email.
- required: true means these fields cannot be empty.
- mongoose.model("users", userSchema) creates the actual "Model" named users (which will create a collection called users in MongoDB) using our userSchema. This userModel is what our User Authentication System uses to interact with user data.

2. The Interview Report Model (Backend/src/models/interviewReport.model.js)

This model defines the more complex structure for storing the AI-generated interview reports.

```
// Backend/src/models/interviewReport.model.js (simplified)
const mongoose = require('mongoose');

// Blueprint for a single technical question (nested inside the main report)
const technicalQuestionSchema = new mongoose.Schema({
   question: { type: String, required: true },
   intention: { type: String, required: true },
   answer: { type: String, required: true }
}, {
   _id: false // This tells Mongoose NOT to automatically create an _id for each question
});

// The main blueprint for an Interview Report
const interviewReportSchema = new mongoose.Schema({
   jobDescription: { type: String, required: true },
   resume: { type: String },
   matchScore: { type: Number, min: 0, max: 100 }, // Score must be 0-100
   technicalQuestions: [technicalQuestionSchema], // This is an array of technical questions
   user: {
      type: mongoose.Schema.Types.ObjectId, // This field stores an ID
      ref: "users" // This ID refers to a document in the "users" collection (our User Model!)
   },
   title: { type: String, required: true }
}, {
   timestamps: true // Automatically adds `createdAt` and `updatedAt` fields
});

const interviewReportModel = mongoose.model("InterviewReport", interviewReportSchema);

module.exports = interviewReportModel;
```

Explanation:

• We define technicalQuestionSchema first, as it's a reusable blueprint for questions *inside* our main report. _id: false prevents Mongoose from adding a unique ID to every single question, as they are part of a larger report.

• interviewReportSchema defines the main structure.

• matchScore has min: 0 and max: 100 rules to ensure the score is always within a valid range.

• technicalQuestions: [technicalQuestionSchema] shows that a report can have many technical questions, all following the technicalQuestionSchema blueprint.

• The user field is special:

  • type: mongoose.Schema.Types.ObjectId says this field will store a unique ID, specifically one that MongoDB uses.

  • ref: "users" is the magic for "referencing." It tells Mongoose that the ID stored here will point

to a document in our users collection (which is managed by our userModel). This creates the link between an interview report and the user who owns it!

• timestamps: true is a handy Mongoose feature that automatically adds createdAt and updatedAt fields to our documents, tracking when they were created and last modified.

3. The Token Blacklist Model (Backend/src/models/blacklist.model.js)

This model, briefly mentioned in Chapter 4: User Authentication System, is a simple blueprint for storing tokens that have been logged out, ensuring they cannot be used again.

```
// Backend/src/models/blacklist.model.js
const mongoose = require('mongoose')

const blacklistTokenSchema = new mongoose.Schema({
    token: {
        type: String,
        required: [true, "token is required to be added in blacklist"]
    }
}, {
    timestamps: true // Also tracks creation time
});

const tokenBlacklistModel = mongoose.model("blacklistTokens", blacklistTokenSchema);

module.exports = tokenBlacklistModel;
```

Explanation:
This is a very simple model. It just needs a token field (which will be a String) and marks it as required. When a user logs out, their JWT is saved here, so our authentication middleware can check against this list.

Conclusion

In this chapter, we've explored **Data Models**, the essential blueprints that define the structure and rules for all the information stored in our database. We learned how Mongoose Schemas act as these blueprints, specifying fields, data types, validation rules, and crucial relationships between different pieces of data, like an Interview Report belonging to a User.

By carefully defining these models, we ensure that our application's data is organized, consistent, and ready to be stored, retrieved, and updated efficiently, forming the robust data foundation for our interview-ai-yt project.